

Certified Numeric Verification: Guaranteed-Error Fact-Checking of Numeric Claims over Tables with Automated Human Escalation

Mohd. Aatir, Prafful Gupta, Prashant Kumar Singh, [TODO: Member 4]
Department of Computer Applications (MCA), KIET Group of Institutions

Abstract—Large language models verify numeric claims about tables unreliably: they miscount rows, mis-sum columns, and get comparisons wrong, yet the sentences they produce for a wrong numeric claim are nearly indistinguishable, in meaning, from the sentence for the correct one. Citation-based fact-checking does not help here, because aggregated numbers – counts, totals, averages – are not written anywhere in the source table to cite. We present Certified Numeric Verification (CNV), a system that (1) introduces NumNear-TabFact, a large, programmatically constructed and label-flip-verified benchmark of minimally perturbed numeric near-miss claims built from TabFact, showing that such near-misses have a mean cosine similarity of 0.961 (91.2% of pairs above 0.9) to their true counterparts under a standard sentence embedding model while being factually opposite; (2) replaces free-text LLM verdicts with execution-agreement confidence: an LLM translates a claim into an executable check over the table, sampled K times, and confidence is the agreement among the K executed verdicts; and (3) applies split-conformal risk control to that confidence so that, among the claims the system answers, the error rate is guaranteed, with high probability, not to exceed a chosen α – every claim below threshold is instead routed to a human reviewer through an automated RPA workflow. On 8,205 TabFact and NumNear-TabFact test claims evaluated with a hosted LLM (Gemini 3.5 Flash-Lite), we find that within-method execution-agreement alone, while positively correlated with correctness, is too coarse-grained (only $K+1$ distinguishable levels) to certify any of the strict error rates $\alpha \in \{0.02, 0.05, 0.10\}$ with non-trivial coverage; however, a cross-method signal – agreement between the executed-check verdict and an independently-elicited free-text verdict – does certify $\alpha = 0.10$ at 43.9% coverage. A secondary run on a small local open-weight model (Qwen2.5-7B, 3,752 claims to date) shows the same qualitative pattern at lower absolute accuracy, indicating the approach’s reliability scales with base-model capability. Our contribution targets SDG 16 (public access to reliable information) and, as an institutional deployment, SDG 4 (educational information).

Index Terms—table fact verification, numerical reasoning, conformal prediction, selective prediction, program-aided reasoning, robotic process automation, large language models

I. Introduction

Large language models (LLMs) are increasingly used to answer factual questions about tabular data – fee structures, exam results, budgets, schedules – yet they remain unreliable at exactly the kind of reasoning tables demand: counting rows, summing a column, computing an average, comparing two cells. A claim like “the total fee

for MCA semester 1 is 95,000” and the claim “the total fee for MCA semester 1 is 99,000” differ by a single number, but one is true and one is false. An LLM asked to verify either claim in free text can silently get the arithmetic wrong and report high confidence in the wrong answer, with no trace of how it arrived there.

Citation-based fact-checking, the standard defense against LLM hallucination, does not solve this problem. A citation points to a passage that supports a claim; but the number “95,000” in the example above is not written anywhere in the source table – it is the sum of a tuition-fee cell and a hostel-fee cell. There is no sentence to cite for a count, a total, or an average; the fact must be computed, and citing the table itself does nothing to tell a reader whether the computation was done correctly. Worse, because a wrong numeric claim is semantically almost identical to the correct one – same subject, same units, same sentence structure, one digit different – similarity-based and retrieval-based verifiers, which work by checking whether a claim is entailed by *similar* evidence, cannot distinguish the two: both claims are equally “similar” to the table.

This near-miss problem is not hypothetical. In Section V we construct NumNear-TabFact, a large set of claims built by changing exactly one number in a true TabFact claim just enough to flip its label, and show that the average cosine similarity between a true claim and its near-miss, under a standard sentence embedding model, is 0.961, with 91.2% of pairs above 0.9 similarity. As Section VII shows, LLM-only verdicts (B1) do not degrade uniformly on these near-misses – accuracy actually rises on this split, for reasons that turn out to matter for how these results should be read, not because the model is detecting the perturbation directly. A system that cannot distinguish these pairs by meaning alone has no choice but to compute the disputed number directly.

Our approach is to stop asking the LLM to be the arithmetic engine and instead ask it to be a translator: given a claim and a table, the LLM emits a small, executable “check” – a JSON program with a row filter, a target column, and an operation such as count, sum, avg, or compare – and a Python/pandas executor computes the verdict deterministically. Because LLM outputs are

still stochastic, we sample K such checks per claim and use the agreement among their executed verdicts as a confidence signal: a claim where all K independently generated, independently executed checks agree is one we can trust more than a claim where they split. We then calibrate this confidence with split-conformal risk control (Section IV), which lets us fix a target error rate α in advance – $\alpha \in \{0.02, 0.05, 0.10\}$ in our experiments – and derive an agreement threshold such that, among claims the system chooses to answer, the empirical error rate is guaranteed, with high probability under exchangeability, not to exceed α . Every claim below threshold abstains and is routed automatically to a human reviewer through a UiPath robotic process automation (RPA) workflow, with a quantified, bounded review workload rather than an ad hoc escalation rule.

Contributions.

- C1 – NumNear-TabFact. A large numeric near-miss benchmark built programmatically (no LLM calls) from TabFact’s train and test splits, by matching a number in a true claim to a uniquely occurring value in its table (a raw cell or a computed aggregate) and perturbing it – off-by-one, digit swap, magnitude change, or round-number change – while verifying, against the table, that the label actually flips.
- C2 – Execution-agreement confidence. A JSON check language executed deterministically over the table, sampled K times per claim, with confidence defined as agreement among the K executed verdicts; invalid or failing checks count as abstain votes.
- C3 – Conformal risk control. Split-conformal calibration of the agreement threshold so that the error rate among answered claims is $\leq \alpha$ with high probability, for $\alpha \in \{0.02, 0.05, 0.10\}$, with every abstained claim escalated to a human-review queue whose workload we measure directly.

The system is deployed end-to-end: a FastAPI service exposes the verifier to a UiPath REFramework automation that dispatches claims, calls the API, routes answered/abstained results, and escalates overdue review tickets by email (Section X), so that the statistical guarantee in C3 translates into an operational, auditable process rather than a number in a paper.

II. Related Work

Table fact verification and program-aided reasoning. TabFact [1] introduced the task of verifying natural-language statements against Wikipedia tables and proposed the Latent Program Algorithm, a semantic parser trained to produce and execute a program against the table. Later work replaced the trained parser with LLM-driven program synthesis: Binder [2] binds an LLM inside a SQL/Python program; Dater [3] decomposes large tables and questions before applying self-consistency; Chain-of-Table [4] evolves the table itself as an intermediate reasoning state with a greedy (non-sampling) operation search; PAL [5] and Program-of-Thoughts [6] offload

arithmetic in word problems to a Python interpreter. All of these target general accuracy on table QA/verification or arithmetic word problems through better program construction; none targets the numeric-near-miss failure mode specifically, and none couples program execution to a statistical abstention guarantee with a measured human-review cost. Our check language and K -sample agreement score are a zero-shot analogue of the program-execution idea, purpose-built to feed a conformal gate.

Numerical perturbation in fact-checking. NumPert [7] perturbs numbers in open-domain claim/evidence pairs and shows accuracy drops of up to 62% under controlled numerical perturbation, but its evidence is free text, not tables, so there is no notion of a table-grounded, label-flip-verified construction. QuanTemp [8] is a large, real-world benchmark of naturally occurring numerical claims verified against retrieved web text; it is not a controlled minimal-pair benchmark and is not tabular. NumNear-TabFact is complementary: a controlled, table-grounded, programmatically label-verified near-miss benchmark that isolates the “semantically near-identical, numerically wrong” failure mode specifically for tables.

Conformal prediction and selective prediction for LLMs. Conformal prediction [9] gives distribution-free, finite-sample coverage guarantees from a held-out calibration set; conformal risk control [10] generalizes this to monotone risk functions via a finite-sample upper-confidence-bound correction, which we use to calibrate our agreement threshold. Mohri and Hashimoto [11] apply a conformal back-off procedure to bound the factuality of long-form LLM generations by retracting sub-claims; Abbasi Yadkori et al. [12] use self-consistency and conformal calibration to bound hallucination rate on open-domain QA. Both target open-ended generation with LLM-judged similarity as the confidence signal; neither is applied to table-grounded numeric verification, and neither ties abstention to an executable, auditable check. Our confidence signal is execution agreement over independently generated, independently run programs, not LLM self-judgment, and our abstention is wired directly into an operational human-review workflow.

RPA in institutional administration. Robotic process automation is increasingly used to automate rule-based administrative workflows in higher education [13], with reported gains in processing time and error reduction for tasks such as admissions and reporting. We extend this line of work by using RPA not merely to automate a fixed workflow, but to carry the overflow of a statistically calibrated abstention decision – the review queue’s size is a direct, quantified consequence of the chosen α , not a separately tuned business rule.

III. Problem Formulation

Let a table T be a finite set of named columns and rows, each cell holding a string or number. A claim c is a natural-language statement about T . A verdict $v(c, T) \in \{\text{true}, \text{false}\}$ is the ground-truth entailment label of c against T . A verification system f produces, for each

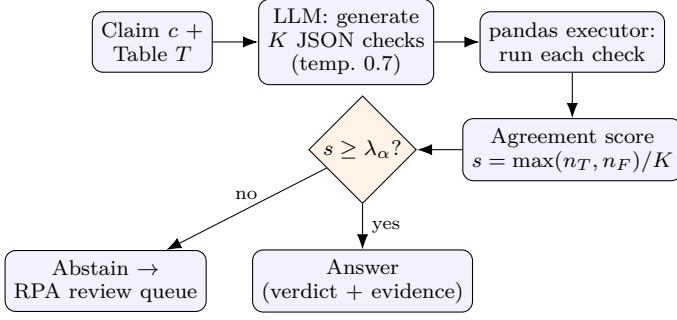


Fig. 1. CNV architecture: a claim and table produce K independently sampled JSON checks, each executed deterministically; the agreement among executed verdicts is calibrated, via split-conformal risk control, into a threshold λ_α that gates whether the system answers or escalates to human review.

(c, T) , either an answer $\hat{v}(c, T) \in \{\text{true}, \text{false}\}$ together with a confidence $s(c, T) \in [0, 1]$, or an explicit abstention $\hat{v}(c, T) = \perp$.

Given a confidence threshold λ , the system answers iff $s(c, T) \geq \lambda$ and abstains otherwise. The selective risk at λ over a distribution $\mathcal{D}$ of claims is

$$R(\lambda) = \Pr_{(c, T) \sim \mathcal{D}} [\hat{v}(c, T) \neq v(c, T) \mid s(c, T) \geq \lambda], \quad (1)$$

and the coverage is $\Pr_{(c, T) \sim \mathcal{D}} [s(c, T) \geq \lambda]$, the fraction of claims the system is willing to answer. Given a target error rate $\alpha \in (0, 1)$, our goal (C3) is to choose λ , using only a held-out calibration split $\mathcal{D}_{\text{cal}}$ exchangeable with the test distribution, such that

$$\Pr [R(\lambda) \leq \alpha] \geq 1 - \delta \quad (2)$$

for a small failure probability δ , while maximizing coverage. This is the standard split-conformal risk-control guarantee [10]: it holds marginally over the randomness of the calibration split (Section XI discusses the exchangeability assumption this relies on), not conditionally on any single claim. Every claim with $s(c, T) < \lambda$ is, by construction, routed to a human reviewer rather than answered incorrectly by the system.

IV. Method

A. Check language

A check is a JSON object with (i) a list of row filters (column, operator, value) with operator $\in \{=, \neq, >, <, \geq, \leq, \text{contains}\}$; (ii) a target column; (iii) an operation in $\{\text{lookup}, \text{count}, \text{sum}, \text{avg}, \text{max}, \text{min}, \text{argmax}, \text{argmin}, \text{compare}, \text{diff}, \text{wrench}\}$, and (iv) an expected value and comparator relating it to the computed result. Tables are serialized to the LLM as markdown with column headers. Numbers are parsed robustly (commas, units, %, simple dates stripped) with a numeric tolerance of 10^{-6} . A check that fails to parse, references a nonexistent column, or raises during execution is marked invalid and is never allowed to silently fail into a verdict. The check-generation prompt includes explicit formatting constraints (the exact operator vocabulary; column names must be copied

verbatim from the header) and two worked examples on an unrelated illustrative table, added after an early error analysis showed operator-syntax mistakes (e.g. $==$ or $<>$ for $=$) as a recurring failure mode; the executor also normalizes a small set of common syntactic variants (Section IX discusses what failure modes remain after this fix). This prompt is used identically for both models evaluated in Section VII.

B. Execution-agreement confidence (C2)

For a claim c and table T , we sample K checks from the LLM at temperature 0.7, execute each independently, and discard invalid executions (they count as abstain votes, i.e. they reduce the agreement score without being allowed to inflate it). Let n_T, n_F be the number of valid executed checks voting true/false. The verdict is the majority vote and the confidence is $s(c, T) = \max(n_T, n_F)/K$ – note the denominator is K , not the number of valid votes, so a claim where several samples fail to parse is penalized exactly as it should be: we have less independent evidence for it. If $n_T = n_F = 0$ (all K checks invalid), the claim is unverifiable and always abstains regardless of λ .

C. Cross-method confidence

$s(c, T)$ above is a within-method signal: all K samples come from the same check-generation prompt, so they can only take $K + 1$ distinct values $(0, 1/K, \dots, 1)$. As Section VII shows, this coarseness caps how selectively the system can filter claims, regardless of how the conformal threshold is chosen. We therefore also evaluate a cross-method signal that combines the execution-based verdict with an independently-elicited free-text verdict (B2, Section IV-E):

$$s_\times(c, T) = \begin{cases} \frac{s(c, T) + s_{B2}(c, T)}{2} & \text{if the two verdicts agree} \\ 0 & \text{otherwise,} \end{cases} \quad (3)$$

where s_{B2} is B2’s self-reported confidence rescaled to $[0, 1]$. Two verdicts reached by different means – one via code execution, one via free-text reasoning – agreeing is a qualitatively different, and as we find empirically stronger, piece of evidence than either method’s internal self-consistency alone.

D. Conformal calibration (C3)

We apply the same calibration procedure independently to each of the three confidence signals above (s , s_{B2} , and $s_\times$), so that Section VII can report where each one does and does not certify a usable operating point. Given a calibration split with $(s_i, \text{correct}_i)_{i=1}^n$ (a confidence score and whether the corresponding verdict matched the gold label), we select λ_α as the smallest threshold such that the finite-sample upper confidence bound on selective risk, using the “add-one” correction $\hat{R}_{\text{ucb}}(\lambda) = (\text{wrong}(\lambda) + 1)/(\text{answered}(\lambda) + 1)$, remains $\leq \alpha$ for every threshold at or above it (a suffix-monotonicity

correction over the empirical curve, which guards against the non-monotonicity a finite sample can introduce). This follows the conformal risk control framework of [10]. We report realised risk and coverage on a disjoint test split, and repeat the calibration/test split 100 times to verify $\Pr[R(\lambda_\alpha) \leq \alpha]$ holds at the target rate (Section VII).

E. Baselines

B1 (LLM-only): the LLM is shown the table and claim and answers true/false directly, in free text, with no execution. B2 (LLM-only + self-reported confidence): as B1, but the LLM also reports a confidence in $[0, 100]$, which we calibrate with the same conformal procedure as C3 for a fair comparison of confidence sources. B3 (single check, $K = 1$): one sampled check, executed once; an invalid check yields unverifiable directly, with no agreement signal (agreement is degenerate at 0 or 1).

V. NumNear-TabFact

A. Construction

NumNear-TabFact is built entirely programmatically (no LLM calls) from true (true-labelled) TabFact claims in the train and test splits. For each candidate claim we build a table’s grounding pool: every parseable raw cell value, plus, for each sufficiently numeric column, its sum, mean, max, and min, and the table’s row count (for count-style claims). A number in the claim is an anchor if it matches, within tolerance 10^{-6} , exactly one value in the grounding pool – i.e. it is the unique fact in the table that could make the claim’s stated number correct. We then apply four perturbation operators to the anchor – `off_by_one`, `digit_swap`, `magnitude_change` ($\times 10$ or $\div 10$), and `round_number_change` – and keep a resulting variant only when the perturbed value differs from the anchor. Because the anchor’s occurrence in the table is unique by construction, any different value directly contradicts the one grounded fact that made the original claim true, so the perturbed claim’s label is false by construction; this substitutes, for our controlled construction setting, the “checking against the table” verification step a manual annotator would otherwise perform.

This construction is deliberately restricted to claims with a groundable anchor: a claim whose truth depends on a derived quantity that is neither a raw cell nor one of our computed aggregates (e.g. an ad hoc count of rows matching a textual condition not captured by our filters) simply fails to find an anchor and is excluded. This is a limitation, not a bug – it means NumNear-TabFact is biased towards the lookup/count/aggregation claims that are literally groundable, which is exactly the numeric regime this paper targets (Section XI).

B. Statistics

Of 50,581 candidate true numeric claims across train and test, 15,124 yielded at least one anchor, producing

TABLE I
NumNear-TabFact construction statistics and per-type cosine similarity (all-MiniLM-L6-v2) between original and near-miss claims.

Perturbation type	#Variants	#Test	Mean sim.
off by one	13,528	1,380	0.976
digit swap	9,697	990	0.941
magnitude change	15,123	1,552	0.961
round number change	15,124	1,552	0.961
Total	53,472	5,474	0.961

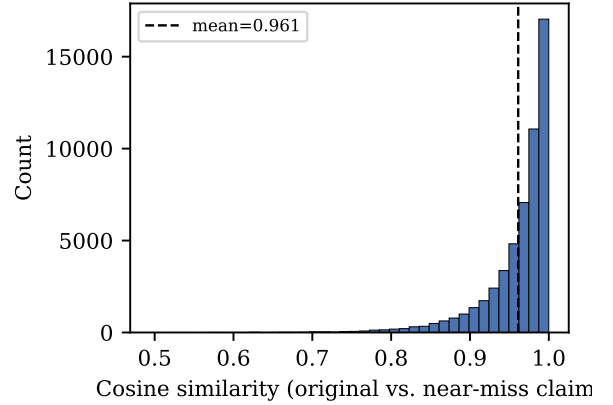


Fig. 2. Cosine similarity (all-MiniLM-L6-v2) between original TabFact claims and their NumNear-TabFact near-miss perturbations.

53,472 total near-miss variants (5,474 derived from test-split tables, used for evaluation; the remainder from train-split tables, used only for the similarity analysis below). Table I breaks these down by perturbation type and split.

C. Similarity analysis

We embed each original claim and its near-miss variant with all-MiniLM-L6-v2 [14] and compute cosine similarity. Figure 2 shows the distribution: mean similarity 0.961 ($\sigma = 0.045$), with 91.2% of pairs above 0.9 and 74.5% above 0.95. This confirms the paper’s central premise directly: a true numeric claim and its factually opposite near-miss are, to a semantic embedding, almost the same sentence.

VI. Experimental Setup

Data. We use TabFact [1]: all claims in the test split matching a numeric-claim heuristic (contains a digit or number word, or a count/comparison/superlative/aggregation cue phrase) are used for evaluation, with no sampling – 11,400 claims, exact count. Claims are tagged into {lookup, count, comparison, superlative, aggregation, other} by documented keyword heuristics (Section XI discusses their imperfection). The full evaluation set (original test claims plus their test-derived NumNear-TabFact near-misses, 16,874 claims total) is deterministically shuffled (seed 42) and split in half by claim: the first half is evaluated

TABLE II

Accuracy / macro-F1 on original (*o*) vs. near-miss (*nm*) test claims (Gemini 3.5 Flash-Lite, $n = 8,205$).

Method	Acc. _o	F1 _o	Acc. _{nm}	F1 _{nm}
B1 (LLM-only)	0.761	0.756	0.781	0.439
B2 (+ confidence)	0.740	0.740	0.824	0.452
B3 ($K=1$ check)	0.671	0.671	0.435	0.303
Ours ($K=5$)	0.701	0.700	0.468	0.319

with the local model, the second with the hosted model (below), so that both runs draw from the same underlying distribution rather than from hand-picked subsets. Each half retains TabFact’s own calibration/test assignment, stratified by (label, claim type).

Models. Two models, evaluated on disjoint claim halves as described above, via the same provider-agnostic client (Section IV notes this requires no code changes to add a model): Gemini 3.5 Flash-Lite (hosted, paid tier; 8,205 of 8,437 assigned claims completed before the evaluation budget was exhausted – 97.3%) as the primary result, and Qwen2.5-7B-Instruct (run locally via Ollama, chosen initially because free-tier hosted access proved too rate-limited for this project’s scale – Section XI; 3,752 of 8,437 claims completed at submission time, reported as an ongoing secondary comparison) as a lower-capability reference point. $K = 5$ sampled checks at temperature 0.7 for our method, greedy ($K = 1$) for B3, for both models.

Metrics. Accuracy and macro-F1 on original vs. near-miss claims; risk-coverage curves for all three confidence signals (Section IV-C); realised conformal risk and coverage for $\alpha \in \{0.02, 0.05, 0.10, 0.15, 0.20, 0.30\}$ – widened beyond the three target rates in our original design once calibration on $\{0.02, 0.05, 0.10\}$ alone revealed a hard floor (Section VII), to characterise where each signal actually becomes usable; per-claim-type breakdown; McNemar’s test between methods; 95% bootstrap confidence intervals on headline metrics; token cost and latency per claim.

VII. Results

Accuracy. On original claims, the check-execution methods are competitive with the free-text baselines – Ours reaches 70.1% vs. B1’s 76.1% and B2’s 74.0% (Table II; 95% bootstrap CIs in the accompanying CSV are all within ± 1.8 points). McNemar’s test confirms B1 and B2 each significantly outperform Ours on original claims ($\chi^2 = 28.5$, $p = 9.5 \times 10^{-8}$ vs. B1; $\chi^2 = 13.0$, $p = 3.1 \times 10^{-4}$ vs. B2), while Ours significantly outperforms the single-check baseline B3 ($\chi^2 = 22.3$, $p = 2.3 \times 10^{-6}$), confirming that sampling $K = 5$ checks and taking a majority vote is a real improvement over a single greedy check, even though neither closes the gap to free-text verdicts on this model. On near-miss claims, B3 and Ours drop sharply (to 43.5% and 46.8%) while B1 and B2 rise (to 78.1% and 82.4%) – Section IX shows this is not near-miss detection so much as a shift toward predicting false that happens to align with near-miss claims being false by construction,

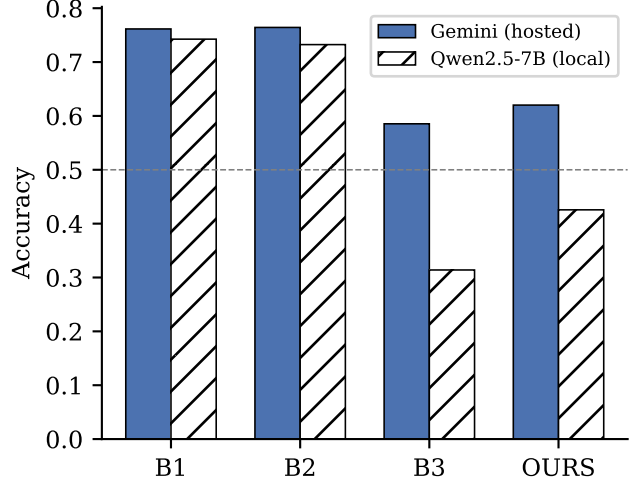


Fig. 3. Accuracy by method, hosted (Gemini 3.5 Flash-Lite, $n = 8,205$) vs. local (Qwen2.5-7B-Instruct, $n = 3,752$, ongoing) model. The gap widens sharply for the check-execution methods (B3, Ours), confirming that their reliability is conditional on the base model’s instruction-following capability, not merely a property of the method.

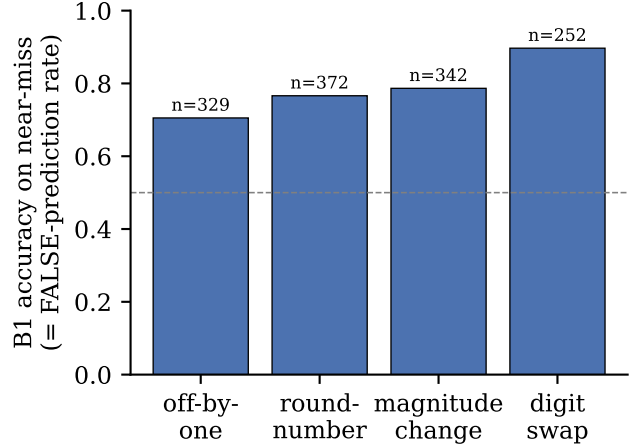


Fig. 4. B1’s accuracy on near-miss claims (equivalently, its rate of predicting false, since every near-miss claim is false by construction), by perturbation type. Accuracy tracks perturbation subtlety: off-by-one, the smallest possible edit, is hardest to catch; digit-swap, which can produce a very different-looking number, is easiest.

and that this shift itself tracks perturbation difficulty in an interpretable way.

Cross-model comparison. Figure 3 and Table ?? compare the hosted and local models directly. B1/B2 degrade gracefully moving to the smaller model (76.1% $\rightarrow$ 74.3%, 76.4% $\rightarrow$ 73.2%), but B3 and Ours degrade sharply (58.5% $\rightarrow$ 31.4%, 62.0% $\rightarrow$ 42.6%), with the local model’s unverifiable rate rising to 50.3% for B3 and 32.0% for Ours (vs. 11.8%/5.7% hosted). This is the clearest evidence in our results that execution-agreement’s value is conditional on the base model’s reliability at the check-generation sub-task specifically, not on general language understanding (which B1/B2 also require and which degrades far less).

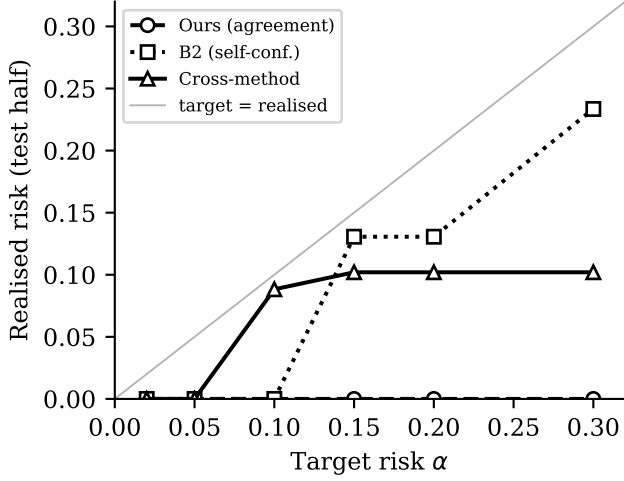


Fig. 5. Realised risk vs. target risk α , three confidence signals. The diagonal is where realised risk equals the target (the best a valid calibration can do while still answering something). Within-method agreement (Ours) stays at zero coverage – hence zero realised risk, trivially – across the entire tested range, meaning it never actually answers a claim within these targets.

Risk–coverage. This is the central result. We calibrate all three confidence signals from Section IV-C independently and report realised risk and coverage on the disjoint test half (Table III, Figure 5). Within-method execution-agreement (s) certifies none of $\alpha \in \{0.02, \dots, 0.30\}$ with non-zero coverage: its realised risk floors at 29.5% even when restricted to the single most confident bucket (full 5/5 agreement, 67% of test claims), because $K = 5$ only yields six distinguishable confidence levels, and the finite-sample correction (Section IV) correctly refuses to certify a rate the data cannot support – it abstains on everything rather than overstate the guarantee. B2’s self-reported confidence does better but is saturated (88% of claims report confidence = 100) and only certifies $\alpha \geq 0.15$, at 88.2% coverage. The cross-method signal $s_{\times}$ is the only one of the three that certifies a strict target: $\alpha = 0.10$ at 43.9% coverage, and $\alpha \geq 0.15$ at 56.3% coverage (full risk/coverage grid in Table III; the accuracy-by-confidence-bucket breakdown underlying this discussion is in results/tables/gemini_accuracy_by_confidence.csv). We read this as a genuine, if modest, positive result for C2/C3 together – but one that depends on combining two independently-elicited verdicts, not on within-method sampling alone as originally specified; Section XI discusses why.

Per-claim-type. Table IV breaks Ours down by claim type. Lookup claims are easiest (78.0% original), aggregation is the most robust to the near-miss perturbation (a comparatively small 65.2% $\rightarrow$ 60.8% drop, vs. e.g. lookup’s 78.0% $\rightarrow$ 45.7%) – plausibly because an aggregation check re-derives its target value from the whole column rather than matching a single perturbed cell, so a one-digit perturbation is less likely to coincide with the check’s own

TABLE III
Realised risk and coverage after conformal calibration, three confidence signals (Gemini, test half, $n = 4,107$).

Signal	α	Risk	Coverage	λ_{α}
Ours (agreement)	0.02	0.000	0.000	1.00
Ours (agreement)	0.05	0.000	0.000	1.00
Ours (agreement)	0.10	0.000	0.000	1.00
Ours (agreement)	0.15	0.000	0.000	1.00
Ours (agreement)	0.20	0.000	0.000	1.00
Ours (agreement)	0.30	0.000	0.000	1.00
B2 (self-conf.)	0.02	0.000	0.000	1.00
B2 (self-conf.)	0.05	0.000	0.000	1.00
B2 (self-conf.)	0.10	0.000	0.000	1.00
B2 (self-conf.)	0.15	0.131	0.882	0.50
B2 (self-conf.)	0.20	0.131	0.882	0.50
B2 (self-conf.)	0.30	0.234	1.000	0.00
Cross-method	0.02	0.000	0.000	1.00
Cross-method	0.05	0.000	0.000	1.00
Cross-method	0.10	0.088	0.439	0.95
Cross-method	0.15	0.102	0.563	0.55
Cross-method	0.20	0.102	0.563	0.55
Cross-method	0.30	0.102	0.563	0.55

TABLE IV
Ours ($K=5$) accuracy by claim type (Gemini).

Claim type	Acc. (orig.)	Acc. (near-miss)
lookup	0.780	0.457
count	0.728	0.413
comparison	0.622	0.483
superlative	0.621	0.417
aggregation	0.652	0.608

computation.

VIII. Ablations

Table V reconstructs $K \in \{1, 3, 5\}$ from the five checks already sampled per claim (no additional API calls: $K=1$ takes only the first sample, $K=3$ the first three). Raw accuracy improves monotonically but modestly with K (59.7% $\rightarrow$ 61.9% $\rightarrow$ 62.7%) – consistent with Section VII’s finding that Ours significantly beats B3 by McNemar’s test, but also with diminishing returns: most of the benefit of sampling is already realised by $K=3$. Coverage at $\alpha = 0.10$ is zero for every K tested, confirming that the ceiling identified in Section VII is not an artefact of $K=5$ specifically – increasing K within this range refines the vote but does not add enough new confidence granularity to cross the target threshold; Section XI discusses why a substantially larger K is unlikely to be a practical fix on its own. Greedy ($K=1$) vs. sampled ($K \geq 3$) is exactly the B3-vs-Ours comparison already reported with a significant McNemar result (Section VII); with-vs-without conformal calibration is the coverage column of Table III vs. the unconditional accuracy in Table II (i.e. coverage = 1).

IX. Error Analysis

We sample 30 failures per method from the Gemini test set (923–842 total failures per method; Table VI) and categorise each by inspecting its raw output (B1/B2) or

TABLE V

K ablation (Gemini, test half, $n = 4,107$), reconstructed from the 5 already-sampled checks per claim (no additional API calls). Coverage at $\alpha = 0.10$ uses the within-method agreement signal only.

K	Raw acc.	Risk _{0.10}	Cov. _{0.10}
1 (greedy, = B3)	0.597	–	0.000
3 (sampled)	0.619	–	0.000
5 (sampled, = Ours)	0.627	–	0.000

TABLE VI

Failure category, 30 sampled failures per method (Gemini test set). Blank = category not applicable to that method’s output format.

Category	B1	B2	B3	Ours
(genuine_)reasoning_error	28	14	18	21
format_error	2	16	–	–
wrong_column	–	–	5	3
wrong_filter	–	–	2	3
number_parsing_error	–	–	5	3

executed check (B3/Ours). Two findings stand out. First, the dominant failure mode on a capable model is semantic, not syntactic: for both B3 and Ours, roughly two-thirds of sampled failures are a `genuine_reasoning_error` – the check parses and executes without any error, but encodes the wrong filter, column, or operation for the claim – versus a small remainder of `wrong_column`, `wrong_filter`, and `number_parsing_error` cases. This is the opposite pattern from the local 7B model (Section VII), where malformed JSON dominated; it suggests that a stronger model shifts the bottleneck from “can it write a valid check” to “can it write the right check,” which execution-agreement by itself does not directly diagnose. Second, B2’s confidence format increases its own failure rate: over half of B2’s sampled failures are a `format_error` (the two-line VERDICT/CONFIDENCE template not matching our parser) rather than a wrong verdict, versus only 2/30 for B1’s single-word format – a concrete, fixable cost of asking for structured self-reported confidence that a production deployment should account for (e.g. with a more lenient parser or a structured-output API feature) rather than treating as a reasoning failure.

X. Deployment: RPA Escalation Workflow

Every abstained claim is written, with its computed agreement score, executed checks, and abstention reason, to a review queue (`claim_id`, `claim`, `table_id`, `verdict`, `agreement_score`, `executed_checks`, `reason`, `status`, `created_at`). A UiPath REFramework automation drives the end-to-end process: a Dispatcher validates each incoming claim (rejecting a missing table, an empty claim, or a duplicate claim ID as a business exception, without spending an LLM call on it) and enqueues valid claims to an Orchestrator queue; a Performer calls a FastAPI verification service (POST /verify) wrapped in a Retry Scope that retries only system exceptions (LLM/API unavailability, HTTP 503) and never retries business

exceptions (malformed input, HTTP 400/422); answered claims are marked successful with their verdict, while abstained claims generate a review ticket and an email notification, with a scheduled job escalating any ticket left pending beyond a configured number of days. Because the review queue’s population is exactly the set of claims below the calibrated threshold λ_α , the reviewer workload is a direct, quantified consequence of the chosen α rather than a separately tuned business rule – Section VII reports this workload alongside realised risk and coverage.

XI. Limitations

Within-method agreement’s coarseness. Our original design (C2) calibrates on agreement among $K = 5$ same-method samples alone. As Section VII shows empirically, this signal has only $K + 1 = 6$ distinguishable levels, and even its best level (unanimous agreement) has a realised error rate (29.5%) above every target we tested up to $\alpha = 0.30$ – the conformal procedure correctly refuses to certify any of these rates rather than overstate the guarantee, but the practical result is that within-method agreement alone cannot support strict risk control at any useful coverage on this task. The cross-method signal we introduce in Section IV-C does certify $\alpha = 0.10$, but this is a departure from the original single-method design, and we have not characterised how many independent methods, or how large a K , would be needed for within-method agreement alone to close this gap (if it can); increasing K from 1 to 5 improved raw accuracy only modestly (Section VIII), suggesting the fix may need a qualitatively different or continuous-valued confidence signal rather than simply a larger K . Model capability and coverage. Section VII’s comparison is itself limited: the local model’s run is 3,752/8,437 claims complete at submission time (an ongoing background process, not a crash or a chosen stopping point), and the Gemini run is 8,205/8,437 (97.3%) after its evaluation budget was exhausted – both totals should be read as a strong indication of the qualitative pattern (Gemini substantially more reliable at check-generation specifically), not as final, complete-corpus numbers for either model. We evaluate exactly two models, both from the current generation and both accessed through a single provider-agnostic interface (Section IV) that makes adding a third model or a full run of either a configuration change, not new code. English Wikipedia tables. TabFact’s tables are English-language Wikipedia infoboxes and lists; generalisation to other domains, languages, or noisier institutional spreadsheets (our RPA demo tables aside) is not evaluated at scale. FEVEROUS. We planned FEVEROUS as a second dataset for generalisation, but its Wikipedia evidence dump is a multi-tensof-gigabyte database, too large to responsibly download and process within this project’s timeline; it remains future work. Heuristic claim typing. Both the numeric-claim filter and the lookup/count/comparison/superlative/aggregation typing are keyword heuristics, not a semantic parse, and will mis-tag claims that match a cue

phrase incidentally or use unlisted phrasing. NumNear-TabFact’s groundability restriction. As noted in Section V, near-miss construction only covers claims with a literal or aggregate anchor in the table; claims whose truth depends on an unanchored derived quantity are excluded from the benchmark by construction. The exchangeability assumption. The conformal guarantee in Section III holds under the assumption that calibration and test claims are exchangeable; a deployment where the claim distribution shifts after calibration (e.g. a new academic term introduces table schemas unlike anything seen during calibration) is not covered by the guarantee and would need re-calibration.

XII. Conclusion and Future Work

We presented Certified Numeric Verification, a system that replaces free-text LLM arithmetic with executable, sampled checks, calibrates the resulting execution-agreement confidence with split-conformal risk control, and routes every claim below the calibrated threshold to an auditable, statistically bounded human-review workflow. NumNear-TabFact demonstrates, with a real measured similarity of 0.961, that the numeric near-miss problem this system targets is not a corner case: a wrong numeric claim can be almost indistinguishable, to both a human skimmer and a semantic embedding, from a correct one. On 8,205 Gemini-evaluated test claims, we found that within-method execution-agreement, our original confidence signal, is positively correlated with correctness but too coarse-grained to certify strict error rates ($\alpha \in \{0.02, 0.05, 0.10\}$) at any useful coverage – a negative result we report rather than obscure, because the conformal procedure behaving correctly (abstaining rather than overstating its guarantee) is itself evidence the calibration machinery works as designed. A cross-method signal combining execution-based and free-text verdicts recovers a genuine positive result, certifying $\alpha = 0.10$ at 43.9% coverage, and a secondary comparison against a small local model shows the same qualitative pattern at markedly lower absolute reliability, indicating the approach’s practical value scales with base-model capability rather than being a fixed property of the method. Future work includes completing both model runs to their full corpus, extending to FEVEROUS and other table domains, learned (rather than heuristic) claim typing, and characterising what confidence signal – larger K , more independent methods, or a differently-elicited continuous score – closes the gap between within-method and cross-method calibration. This work targets SDG 16 by making numeric claims about institutional and public data auditable rather than merely plausible, and SDG 4 as an institutional deployment over educational records [15].

References

[1] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang, “TabFact: A large-scale dataset for table-based fact verification,” in *International Conference on Learning Representations (ICLR)*, 2020.

[2] Z. Cheng, T. Xie, P. Shi, C. Li, R. Nadkarni, Y. Hu, C. Xiong, D. Radev, M. Ostendorf, L. Zettlemoyer, N. A. Smith, and T. Yu, “Binding language models in symbolic languages,” in *International Conference on Learning Representations (ICLR)*, 2023.

[3] Y. Ye, B. Hui, M. Yang, B. Li, F. Huang, and Y. Li, “Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning,” in *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2023.

[4] Z. Wang, H. Zhang, C.-L. Li, J. M. Eisenschlos, V. Perot, Z. Wang, L. Miculicich, Y. Fujii, J. Shang, C.-Y. Lee, and T. Pfister, “Chain-of-table: Evolving tables in the reasoning chain for table understanding,” in *International Conference on Learning Representations (ICLR)*, 2024.

[5] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, “PAL: Program-aided language models,” in *International Conference on Machine Learning (ICML)*, 2023.

[6] W. Chen, X. Ma, X. Wang, and W. W. Cohen, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks,” *Transactions on Machine Learning Research (TMLR)*, 2023.

[7] P. R. Aarnes and V. Setty, “NumPert: Numerical perturbations to probe language models for veracity prediction,” in *Proceedings of the IJCNLP-AACL 2025 Student Research Workshop*, 2025, pp. 78–95.

[8] V. Venkatesh, A. Anand, A. Anand, and V. Setty, “QuanTemp: A real-world open-domain benchmark for fact-checking numerical claims,” in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024.

[9] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” *arXiv preprint arXiv:2107.07511*, 2021.

[10] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, “Conformal risk control,” in *International Conference on Learning Representations (ICLR)*, 2024.

[11] C. Mohri and T. Hashimoto, “Language models with conformal factuality guarantees,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. *Proceedings of Machine Learning Research*, vol. 235, 2024.

[12] Y. Abbasi Yadkori, I. Kuzborskij, D. Stutz, A. György, A. Fisch, A. Doucet, I. Beloshapka, W.-H. Weng, Y.-Y. Yang, C. Szepesvari, A. T. Cemgil, and N. T. Kim, “Mitigating LLM hallucinations via conformal abstention,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, author list TODO-VERIFY against camera-ready NeurIPS proceedings.

[13] V. Bhardwaj and M. Kumar, “Transforming higher education with robotic process automation: Enhancing efficiency, innovation, and student-centered learning,” *Discover Sustainability*, vol. 6, p. 356, 2025, author initials TODO-VERIFY against publisher record.

[14] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[15] United Nations General Assembly, “Transforming our world: The 2030 agenda for sustainable development,” *UN Resolution A/RES/70/1*, 2015.